Movie Ticket Booking System

Overview:

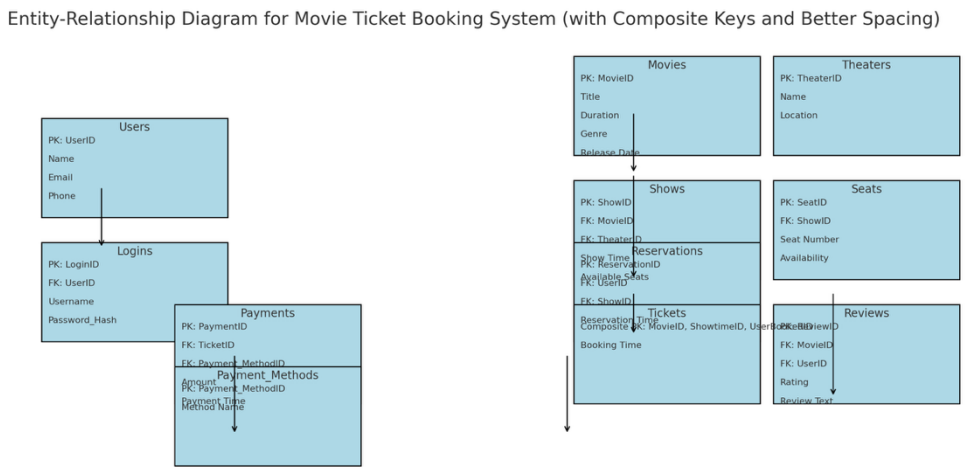
This system allows users to browse movie schedules, book tickets, select seats, make payments, and leave reviews. Each ticket booking is uniquely identified by a composite key that combines the **movie_id**, **showtime_id**, and **userbooked_id**. Users can log in to the system, and payments are tied to specific bookings.

Key Features:

- **Composite Key**: Uniquely identifies each ticket using `movie_id-showtime_id-userbooked_id`.
- **Login System**: Users log in using a secure username and password.
- **Seat Reservations**: Users select seats that are managed by availability.
- **Payment System**: Supports multiple payment methods for booking tickets.
- **Movie Reviews**: Users can leave feedback and rate movies.

Database Entities:

- **Users, Logins, Theaters, Movies, Shows, Seats, Reservations, Tickets, Payments, Reviews, Payment Methods**



Database Tables:

- Users (UserID, Name, Email, PasswordHash)
- Movies (MovieID, Title, Genre, Duration)
- Theaters (TheaterID, Name, Location)
- Showtimes (ShowtimeID, MovieID, TheaterID, Time)
- Seats (SeatID, ShowtimeID, SeatNumber, Availability)
- Reservations (ReservationID, UserID, ShowtimeID, ReservationTime)
- Tickets (Composite Key: MovieID, ShowtimeID, UserBookedID, BookingTime)
- Payments (PaymentID, TicketID, PaymentMethodID, Amount)
- Reviews (ReviewID, MovieID, UserID, Rating, ReviewText)
- Payment Methods (PaymentMethodID, MethodName)

Database Schema

```
CREATE TABLE Users (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(255) NOT NULL,
    Email NVARCHAR(255) UNIQUE NOT NULL,
    Phone NVARCHAR(20),
    PasswordHash NVARCHAR(255) NOT NULL
);

CREATE TABLE Logins (
    LoginID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    Username NVARCHAR(255) UNIQUE NOT NULL,
    PasswordHash NVARCHAR(255) NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

CREATE TABLE Movies (
    MovieID INT IDENTITY(1,1) PRIMARY KEY,
    Title NVARCHAR(255) NOT NULL,
    Genre NVARCHAR(100),
    Duration INT,  -- In minutes
    ReleaseDate DATE
```

```
);

CREATE TABLE Theaters (
    TheaterID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(255) NOT NULL,
    Location NVARCHAR(255) NOT NULL
);

CREATE TABLE Showtimes (
    ShowtimeID INT IDENTITY(1,1) PRIMARY KEY,
    MovieID INT,
    TheaterID INT,
    ShowTime DATETIME NOT NULL,
    AvailableSeats INT NOT NULL,
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (TheaterID) REFERENCES Theaters(TheaterID)
);

CREATE TABLE Seats (
    SeatID INT IDENTITY(1,1) PRIMARY KEY,
    ShowtimeID INT,
    SeatNumber INT NOT NULL,
    IsAvailable BIT DEFAULT 1,
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID)
);

CREATE TABLE Reservations (
    ReservationID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    ShowtimeID INT,
    ReservationTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID)
);

CREATE TABLE Tickets (
    MovieID INT,
    ShowtimeID INT,
    UserBookedID INT,
```

```
    BookingTime DATETIME DEFAULT GETDATE(),
    PRIMARY KEY (MovieID, ShowtimeID, UserBookedID),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID),
    FOREIGN KEY (UserBookedID) REFERENCES Users(UserID)
);

CREATE TABLE PaymentMethods (
    PaymentMethodID INT IDENTITY(1,1) PRIMARY KEY,
    MethodName NVARCHAR(100) NOT NULL
);

CREATE TABLE Payments (
    PaymentID INT IDENTITY(1,1) PRIMARY KEY,
    TicketID INT,
    PaymentMethodID INT,
    Amount DECIMAL(10, 2) NOT NULL,
    PaymentTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (TicketID) REFERENCES Tickets(MovieID, ShowtimeID,
UserBookedID),
    FOREIGN KEY (PaymentMethodID) REFERENCES
PaymentMethods(PaymentMethodID)
);

CREATE TABLE Reviews (
    ReviewID INT IDENTITY(1,1) PRIMARY KEY,
    MovieID INT,
    UserID INT,
    Rating INT CHECK (Rating >= 1 AND Rating <= 5),
    ReviewText NVARCHAR(MAX),
    ReviewDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);
```

UPDATED SCRIPT WITH ERRORS REMOVED -

```sql
-- Use the MovieTheatre database
USE MovieTheatre;

-- Safely drop foreign key constraints only if they exist
IF OBJECT_ID('Logins', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Logins_Users')
            ALTER TABLE Logins DROP CONSTRAINT FK_Logins_Users;
    END

IF OBJECT_ID('Showtimes', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Showtimes_Movies')
            ALTER TABLE Showtimes DROP CONSTRAINT FK_Showtimes_Movies;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Showtimes_Theaters')
            ALTER TABLE Showtimes DROP CONSTRAINT FK_Showtimes_Theaters;
    END

IF OBJECT_ID('Seats', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Seats_Showtimes')
            ALTER TABLE Seats DROP CONSTRAINT FK_Seats_Showtimes;
    END

IF OBJECT_ID('Reservations', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Reservations_Users')
            ALTER TABLE Reservations DROP CONSTRAINT FK_Reservations_Users;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Reservations_Showtimes')
            ALTER TABLE Reservations DROP CONSTRAINT FK_Reservations_Showtimes;
    END

IF OBJECT_ID('Tickets', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Tickets_Movies')
            ALTER TABLE Tickets DROP CONSTRAINT FK_Tickets_Movies;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Tickets_Showtimes')
            ALTER TABLE Tickets DROP CONSTRAINT FK_Tickets_Showtimes;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Tickets_Users')
            ALTER TABLE Tickets DROP CONSTRAINT FK_Tickets_Users;
    END

IF OBJECT_ID('Payments', 'U') IS NOT NULL
```

```sql
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Payments_Tickets')
            ALTER TABLE Payments DROP CONSTRAINT FK_Payments_Tickets;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Payments_PaymentMethods')
            ALTER TABLE Payments DROP CONSTRAINT FK_Payments_PaymentMethods;
    END

IF OBJECT_ID('Reviews', 'U') IS NOT NULL
    BEGIN
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Reviews_Movies')
            ALTER TABLE Reviews DROP CONSTRAINT FK_Reviews_Movies;
        IF EXISTS (SELECT * FROM sys.foreign_keys WHERE name = 'FK_Reviews_Users')
            ALTER TABLE Reviews DROP CONSTRAINT FK_Reviews_Users;
    END

-- Drop tables if they already exist to avoid conflicts
DROP TABLE IF EXISTS Payments;
DROP TABLE IF EXISTS PaymentMethods;
DROP TABLE IF EXISTS Tickets;
DROP TABLE IF EXISTS Reservations;
DROP TABLE IF EXISTS Seats;
DROP TABLE IF EXISTS Showtimes;
DROP TABLE IF EXISTS Theaters;
DROP TABLE IF EXISTS Movies;
DROP TABLE IF EXISTS Logins;
DROP TABLE IF EXISTS Users;
DROP TABLE IF EXISTS Reviews;

-- Create Users table
CREATE TABLE Users (
    UserID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(255) NOT NULL,
    Email NVARCHAR(255) UNIQUE NOT NULL,
    Phone NVARCHAR(20),
    PasswordHash NVARCHAR(255) NOT NULL
);

-- Create Logins table
CREATE TABLE Logins (
    LoginID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    Username NVARCHAR(255) UNIQUE NOT NULL,
    PasswordHash NVARCHAR(255) NOT NULL,
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

-- Create Movies table
CREATE TABLE Movies (
    MovieID INT IDENTITY(1,1) PRIMARY KEY,
    Title NVARCHAR(255) NOT NULL,
    Genre NVARCHAR(100),
    Duration INT,  -- In minutes
    ReleaseDate DATE
);

-- Create Theaters table
CREATE TABLE Theaters (
    TheaterID INT IDENTITY(1,1) PRIMARY KEY,
    Name NVARCHAR(255) NOT NULL,
```

```sql
    Location NVARCHAR(255) NOT NULL
);

-- Create Showtimes table
CREATE TABLE Showtimes (
    ShowtimeID INT IDENTITY(1,1) PRIMARY KEY,
    MovieID INT,
    TheaterID INT,
    ShowTime DATETIME NOT NULL,
    AvailableSeats INT NOT NULL,
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (TheaterID) REFERENCES Theaters(TheaterID)
);

-- Create Seats table
CREATE TABLE Seats (
    SeatID INT IDENTITY(1,1) PRIMARY KEY,
    ShowtimeID INT,
    SeatNumber INT NOT NULL,
    IsAvailable BIT DEFAULT 1,
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID)
);

-- Create Reservations table
CREATE TABLE Reservations (
    ReservationID INT IDENTITY(1,1) PRIMARY KEY,
    UserID INT,
    ShowtimeID INT,
    ReservationTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID)
);

-- Create Tickets table with a new TicketID as the primary key
CREATE TABLE Tickets (
    TicketID INT IDENTITY(1,1) PRIMARY KEY,  -- New primary key
    MovieID INT,
    ShowtimeID INT,
    UserBookedID INT,
    BookingTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID),
    FOREIGN KEY (UserBookedID) REFERENCES Users(UserID)
);

-- Create PaymentMethods table
CREATE TABLE PaymentMethods (
    PaymentMethodID INT IDENTITY(1,1) PRIMARY KEY,
    MethodName NVARCHAR(100) NOT NULL
);

-- Create Payments table with the correct foreign key reference to TicketID
CREATE TABLE Payments (
    PaymentID INT IDENTITY(1,1) PRIMARY KEY,
    TicketID INT,  -- Foreign key now references the new TicketID
    PaymentMethodID INT,
    Amount DECIMAL(10, 2) NOT NULL,
    PaymentTime DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (TicketID) REFERENCES Tickets(TicketID),
```

```sql
    FOREIGN KEY (PaymentMethodID) REFERENCES PaymentMethods(PaymentMethodID)
);

-- Create Reviews table
CREATE TABLE Reviews (
    ReviewID INT IDENTITY(1,1) PRIMARY KEY,
    MovieID INT,
    UserID INT,
    Rating INT CHECK (Rating >= 1 AND Rating <= 5),
    ReviewText NVARCHAR(MAX),
    ReviewDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
    FOREIGN KEY (UserID) REFERENCES Users(UserID)
);

-- Insert sample data into Users table
INSERT INTO Users (Name, Email, Phone, PasswordHash) VALUES
('John Doe', 'john@example.com', '123-456-7890', 'hashedpassword1'),
('Jane Smith', 'jane@example.com', '987-654-3210', 'hashedpassword2'),
('Michael Johnson', 'michael@example.com', '555-555-5555', 'hashedpassword3');

-- Insert sample data into Logins table
INSERT INTO Logins (UserID, Username, PasswordHash) VALUES
(1, 'john_doe', 'hashedpassword1'),
(2, 'jane_smith', 'hashedpassword2'),
(3, 'michael_johnson', 'hashedpassword3');

-- Insert sample data into Movies table
INSERT INTO Movies (Title, Genre, Duration, ReleaseDate) VALUES
('The Matrix', 'Sci-Fi', 136, '1999-03-31'),
('Inception', 'Sci-Fi', 148, '2010-07-16'),
('The Dark Knight', 'Action', 152, '2008-07-18');

-- Insert sample data into Theaters table
INSERT INTO Theaters (Name, Location) VALUES
('Downtown Cinema', '123 Main St'),
('Uptown Theater', '456 High St'),
('City Center Cinemas', '789 Center Blvd');

-- Insert sample data into Showtimes table
INSERT INTO Showtimes (MovieID, TheaterID, ShowTime, AvailableSeats) VALUES
(1, 1, '2024-10-07 18:30:00', 50),
(2, 2, '2024-10-07 20:00:00', 40),
(3, 3, '2024-10-08 15:00:00', 60);

-- Insert sample data into Seats table
INSERT INTO Seats (ShowtimeID, SeatNumber, IsAvailable) VALUES
(1, 1, 1), (1, 2, 1), (1, 3, 1),
(2, 1, 1), (2, 2, 1), (2, 3, 1),
(3, 1, 1), (3, 2, 1), (3, 3, 1);

-- Insert sample data into Reservations table
INSERT INTO Reservations (UserID, ShowtimeID) VALUES
(1, 1),
(2, 2),
(3, 3);
```

```sql
-- Insert sample data into Tickets table
INSERT INTO Tickets (MovieID, ShowtimeID, UserBookedID) VALUES
(1, 1, 1),  -- John Doe booked for The Matrix
(2, 2, 2),  -- Jane Smith booked for Inception
(3, 3, 3);  -- Michael Johnson booked for The Dark Knight

-- Insert sample data into PaymentMethods table
INSERT INTO PaymentMethods (MethodName) VALUES
('Credit Card'),
('PayPal'),
('Bank Transfer');

-- Insert sample data into Payments table
INSERT INTO Payments (TicketID, PaymentMethodID, Amount) VALUES
(1, 1, 15.00),  -- John paid with Credit Card
(2, 2, 12.50),  -- Jane paid with PayPal
(3, 3, 18.00);  -- Michael paid with Bank Transfer

-- Insert sample data into Reviews table
INSERT INTO Reviews (MovieID, UserID, Rating, ReviewText) VALUES
(1, 1, 5, 'Amazing movie with groundbreaking visual effects!'),
(2, 2, 4, 'Mind-bending thriller with a complex plot.'),
(3, 3, 5, 'One of the best superhero movies ever made.');

-- Down script to drop all the tables
DROP TABLE IF EXISTS Reviews;
DROP TABLE IF EXISTS Payments;
DROP TABLE IF EXISTS PaymentMethods;
DROP TABLE IF EXISTS Tickets;
DROP TABLE IF EXISTS Reservations;
DROP TABLE IF EXISTS Seats;
DROP TABLE IF EXISTS Showtimes;
DROP TABLE IF EXISTS Theaters;
DROP TABLE IF EXISTS Movies;
DROP TABLE IF EXISTS Logins;
DROP TABLE IF EXISTS Users;
```

Composite Key:

```sql
CREATE TABLE Tickets (
   MovieID INT,
   ShowtimeID INT,
   UserBookedID INT,
   BookingTime DATETIME DEFAULT GETDATE(),
   PRIMARY KEY (MovieID, ShowtimeID, UserBookedID),
   FOREIGN KEY (MovieID) REFERENCES Movies(MovieID),
   FOREIGN KEY (ShowtimeID) REFERENCES Showtimes(ShowtimeID),
   FOREIGN KEY (UserBookedID) REFERENCES Users(UserID)
);
```

Payment System:

```
-- Create PaymentMethods Table
CREATE TABLE PaymentMethods (
   PaymentMethodID INT IDENTITY(1,1) PRIMARY KEY,
   MethodName NVARCHAR(100) NOT NULL
);

-- Insert Initial Payment Methods
INSERT INTO PaymentMethods (MethodName) VALUES ('Credit Card'), ('Debit Card'),
('PayPal'), ('Apple Pay');

-- Create Payments Table
CREATE TABLE Payments (
   PaymentID INT IDENTITY(1,1) PRIMARY KEY,
   TicketID INT,
   PaymentMethodID INT,
   Amount DECIMAL(10, 2) NOT NULL,
   PaymentTime DATETIME DEFAULT GETDATE(),
   FOREIGN KEY (TicketID) REFERENCES Tickets(MovieID, ShowtimeID, UserBookedID),
   FOREIGN KEY (PaymentMethodID) REFERENCES PaymentMethods(PaymentMethodID)
);

-- Sample SQL Transaction for Handling Payments
BEGIN TRANSACTION;

-- Insert Reservation (assuming relevant variables are provided)
INSERT INTO Reservations (UserID, ShowtimeID) VALUES (@UserID, @ShowtimeID);

-- Insert Payment (assuming relevant variables are provided)
INSERT INTO Payments (TicketID, PaymentMethodID, Amount) VALUES (@TicketID,
@PaymentMethodID, @Amount);

-- Update Seat Availability (assuming relevant variables are provided)
UPDATE Seats
SET IsAvailable = 0
WHERE SeatID = @SeatID;
```

COMMIT;